

Monitoring EHPAD : architecture et écarts

Surveillance temps réel de 20 résidents : capteurs simulés, ingestion MQTT, scoring ML, alertes à cinq niveaux avec auto-escalade, détection de fugue, résumé quotidien par LLM local. Voici l'architecture telle qu'elle est, puis ce qui manque pour en faire un produit utilisable dans un établissement.

9 services Docker · ~4 700 lignes · 59 tests backend, 0 côté front ·
74 commits, avril 2026

01

Architecture



MQTT fait la colonne vertébrale : le simulateur se remplace par de vrais capteurs sans toucher au backend. Une seule origine côté navigateur, d'où l'absence de

configuration CORS dans le code. Les neuf services démarrent par un unique `docker compose up`.

Écarts avec un produit fini

Dix points manquants, cotés en criticité et en effort.

	ÉCART	CRITICITÉ	EFFORT
É1	Aucun capteur réel Toute la donnée vient du simulateur. Aucun pilote BLE ou LoRa, pas d'appairage, pas de gestion de batterie ni de perte de lien.	BLOQUANT	Élevé
É2	L'alerte ne sort pas du navigateur Un toast dans l'onglet ouvert, rien d'autre. La nuit, personne devant l'écran : il faut du SMS, du push ou le bip de l'établissement.	BLOQUANT	Moyen
É3	Aucune authentification Toutes les routes sont ouvertes. N'importe qui lit les constantes des 20 résidents et peut acquitter — donc éteindre — une alerte vitale.	BLOQUANT	Moyen
É4	MQTT anonyme, aucun TLS <code>allow_anonymous true</code> et zéro chiffrement. Un faux client peut injecter de fausses constantes.	BLOQUANT	Moyen
É5	Résidents et personnel codés en dur Déclarés en Python. Admettre un résident ou le changer de chambre demande un redéploiement.	BLOQUANT	Moyen
É6	ML entraîné sur nos propres données simulées Le modèle apprend le simulateur, pas la physiologie. Aucune mesure de faux positifs, donc rien de démontrable sur la détection.	MAJEUR	Élevé
É7	Un seul backend, escalades en mémoire S'il redémarre, les escalades en cours disparaissent sans bruit : une alerte non acquittée ne montera jamais.	MAJEUR	Élevé
É8	Données éphémères, aucune sauvegarde Volumes Docker locaux, rétention Influx de 30 jours. Aucune trace de qui a acquitté quelle alerte.	MAJEUR	Moyen
É9	Hébergement non HDS Des données de santé nominatives imposent un hébergeur certifié HDS. Chantier d'organisation, pas de code.	MAJEUR	Hors dev
É10	Pas de CI, aucun test front 59 tests backend solides, mais rien sur React ni sur la chaîne capteur → alerte → écran, et rien ne s'exécute automatiquement.	MINEUR	Faible

Bloquant = empêche la mise en service Majeur = exploitation dégradée Mineur = dette acceptable
Faible ≤ 2 j · Moyen 3–8 j · Élevé > 8 j

03

Ce qu'il faut en retenir

La chaîne fonctionne de bout en bout : un capteur publie, l'alerte arrive à l'écran en moins de deux secondes, l'escalade s'enclenche seule, le ML anticipe une dégradation. Comme démonstrateur, c'est complet.

Ce qui manque n'est presque jamais une fonctionnalité. Ce sont les capteurs physiques, une identité derrière chaque geste, et une alerte qui franchit les limites du navigateur. Tant que ces trois-là manquent, un établissement ne peut pas brancher le système.

1. **D'abord** — É2 et É3 : faire sortir l'alerte de l'écran, et savoir qui l'acquitte.
2. **Ensuite** — É4, É5, É8 : chiffrement, référentiels en base, sauvegardes.
3. **Enfin** — É1 et É6 : vrais capteurs et validation du modèle, les deux gros chantiers.